Generative AI Models

An Introduction

CS3263

Foundations of Artificial Intelligence

Version 1.0.1

Topics

- Generative AI models
  - Probabilistic generative models
  - Deep generative models
- Variational auto-encoder
- Diffusion models
- Large language models

Generative AI Models

Probabilistic generative models
Deep generative models

Generative Models

- Definitions:
  - A generative model is a joint probability distribution $P(\boldsymbol{y})$, for $\boldsymbol{y} \in \boldsymbol{Y}$.
  - The model may be conditioned on inputs or covariates $\boldsymbol{x} \in \boldsymbol{X}$, which gives rise to a conditional generative model of the form $P(\boldsymbol{y}|\boldsymbol{x})$.
- Types of generative models
  - Probabilistic graphical models (PGM)
  - Deep generative models (DGM)
  - Hybrids are possible

Ref: Murphy, K. 2023, Book 2 Chapter 20

Probabilistic Graphical Models

- Bayesian networks (BNs) – Directed PGMs
  - Consider a set of random variables $\boldsymbol{X} = \{X_1, X_2, \dots, X_n\}$
  - In BNs, each node X_i is conditionally independent of its non-descendants given its parent nodes.
  - The model parameters typically consist of the local conditional probability distributions

$$P(X_1, X_2, \dots, X_n) = \prod_{i=1}^{n} P(X_i \mid Pa(X_i))$$

where $Pa(X_i)$ are the parent variables of X_i in the graph.

Probabilistic Graphical Models

- Markov networks or Markov random fields (MRFs) – Undirected PGMs
  - Consider a set of random variables $\boldsymbol{X} = \{X_1, X_2, \dots, X_n\}$ and a set of potential functions Ψ – measure compatibility of different configurations of these variables.
  - Joint distribution over state space $\boldsymbol{x}$ of the random variables can be written as:

$$P(\mathbf{x}) = \frac{1}{Z} \prod_{c \in C} \Psi_c(\mathbf{x}_c)$$

Summing or integrating over all possible states of random variables:

    - C is the set of cliques (fully connected subgraphs) in the graph.
    - $\Psi_c(\boldsymbol{x}_c)$ are the potential functions that only depend on the variables within clique c.
    - $\boldsymbol{x}_c$ is the configuration of variables in clique c.
    - Z is the partition function – normalization factor ensuring probabilities sum up to 1:

$$Z = \sum_{\mathbf{x}} \prod_{c \in C} \Psi_c(\mathbf{x}_c)$$

Probabilistic Graphical Models

- Both Bayesian networks (BN) and Markov random fields (MRFs) can be used as generative models:
  1. Specifying:
  - conditional probabilities (for BNs) or
  - potential functions (for MRFs)
  2. Sampling:
  - generate instances from the joint distribution.
- Main Applications:
  - BNs: causal reasoning and situations where the direction of influence is essential
  - MRFs: modeling undirected dependencies e.g., spatial modelling for image analysis.

Deep Generative Models

- Variational autoencoders (VAEs)
- Autoregressive models (ARMs)
- Normalizing flows
- Diffusion models
- Energy based models (EBMs)
- Generative adversarial networks (GANs)
- Etc.

Ref: Murphy, K. (2023) Book 2

Deep Generative Models

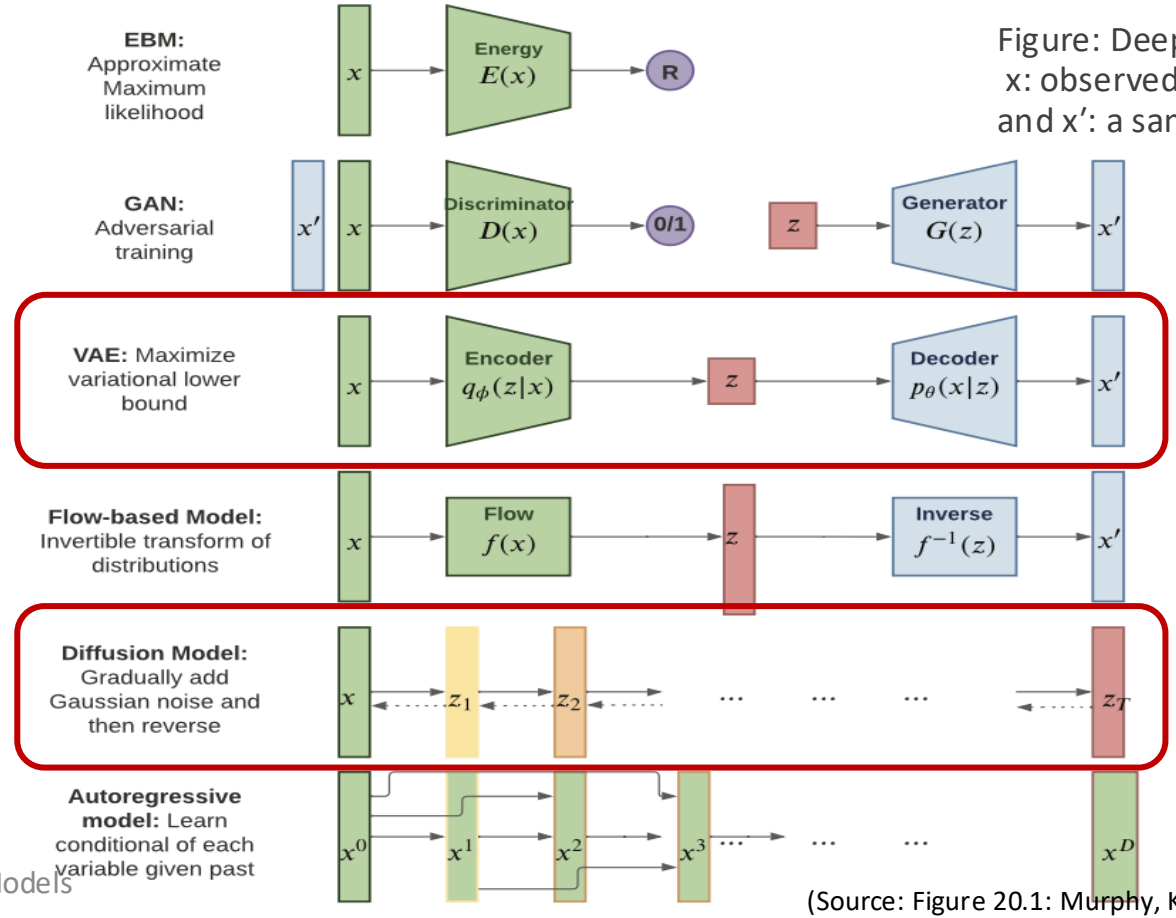


Figure: Deep generative models. x: observed data, z : latent code, and x′: a sample from the model.

(Source: Figure 20.1: Murphy, K. 2023, Book 2)

Characteristics of Generative Models

Model	Chapter	Density	Sampling	Training	Latents	Architecture
PGM-D	Section 4.2	Exact, fast	Fast	MLE	Optional	Sparse DAG
PGM-U	Section 4.3	Approx, slow	Slow	MLE-A	Optional	Sparse graph
VAE	Chapter 21	LB, fast	Fast	MLE-LB	$\mathbb{R}^L$	Encoder-Decoder
ARM	Chapter 22	Exact, fast	Slow	MLE	None	Sequential
Flows	Chapter 23	Exact, slow/fast	Slow	MLE	$\mathbb{R}^D$	Invertible
EBM	Chapter 24	Approx, slow	Slow	MLE-A	Optional	Discriminative
Diffusion	Chapter 25	LB	Slow	MLE-LB	$\mathbb{R}^D$	Encoder-Decoder
GAN	Chapter 26	NA	Fast	Min-max	$\mathbb{R}^L$	Generator-Discriminator

Table 20.1: Characteristics of common kinds of generative model.

D: dimensionality of the observed x, and L: dimensionality of the latent z, if present. (Usually assume $L \ll D$, although overcomplete representations can have $L \gg D$.)
Approx = approximate, ARM = autoregressive model, EBM = energy based model, GAN = generative adversarial network, MLE = maximum likelihood estimation, MLE-A = MLE (approximate), MLE-LB = MLE (lower bound), NA = not available, PGM = probabilistic graphical model, PGM-D = directed PGM, PGM-U = undirected PGM, VAE = variational autoencoder.

(Source: Murphy, K. 2023, Book 2)

Variational Auto-Encoder

Review: Inverse Transform Sampling
Variational Auto-Encoder (VAE)
Conditional Variational Auto-Encoder (CVAE)

Inverse Transform Sampling

- Generating from simple distributions are straightforward:
  - Normal distribution: **np.random.normal(1.5, 2.0, 1000)**
  - Gamma distribution: **np.random.gamma(5.0, 2.0, 1000)**
  - Exponential distribution: **np.random.exponential(5.0, 1000)**

Generating from these distributions are supported by many programming language.

What if we are working on an environment or programming language that does not support these functions?
We can use inverse transform sampling if the distribution is known.

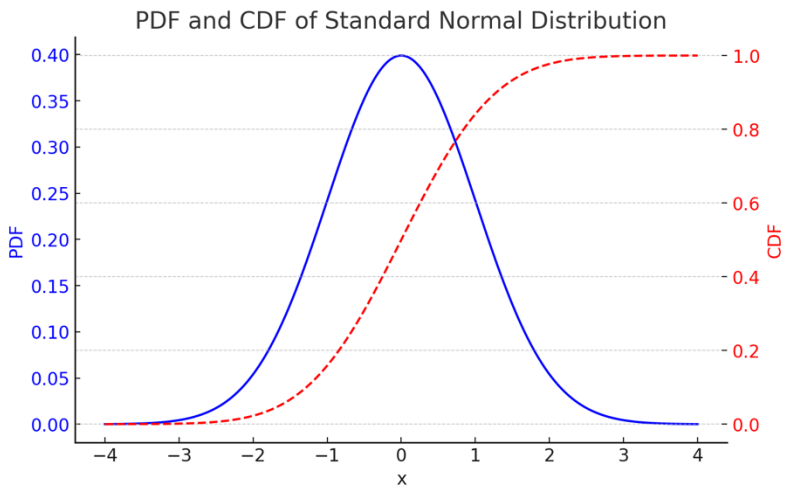
Inverse Transform Sampling

Given a simple distribution such as Gaussian, Gamma, or Exponential, we know their probability density function (PDF) and Cumulative distribution function (CDF)

For example: Gaussian distribution

PDF: $$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

CDF: $$F(x) = \frac{1}{2}\left[1 + \mathrm{erf}\left(\frac{x-\mu}{\sigma\sqrt{2}}\right)\right]$$



Inverse Transform Sampling

For any random variable X we have $F(X) \sim U[0,1]$

$$F(X) = \frac{1}{2}\left(1 + \mathrm{erf}\left(\frac{X-\mu}{\sigma\sqrt{2}}\right)\right), \qquad F(X) \sim U[0,1]$$

$$X = \sigma\sqrt{2}\mathrm{erf}^{-1}(2F(X) - 1), \qquad F(X) \sim U[0,1]$$

Equivalently,

$$X = \sigma\sqrt{2}\mathrm{erf}^{-1}(2Z - 1), \qquad Z \sim U[0,1]$$

Inverse Transform Sampling

$$X = \sigma\sqrt{2}\mathrm{erf}^{-1}(2Z - 1)\,, \qquad Z \sim U[0,1]$$

Algorithm:

1. Draw a sample z from $U[0,1]$
2. Calculate $x = \sigma\sqrt{2}\mathrm{erf}^{-1}(2z - 1)$

This method can only be applied on simple distributions where we know their CDF.

How to draw samples from more complex data such as images, text, videos where we do not know their PDF, CDF?

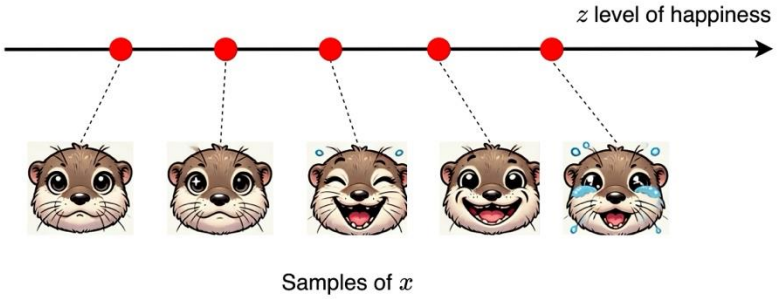
Variational Auto-Encoder (VAE)

The problem:

- We want to draw $x \sim p(x)$.
- We do not know PDF and CDF of x.
- We have data samples of x.

The idea: each object is associated with a latent variable z

$$p(x) = \int_z p(x|z)p(z)dz$$

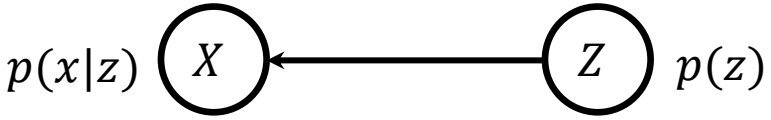


Variational Auto-Encoder (VAE)

The idea: each object is associated with a latent variable z

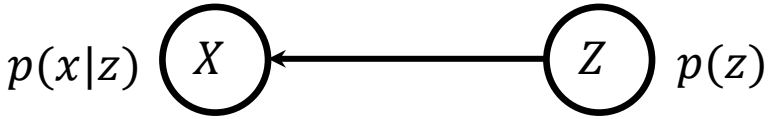
$$p(x) = \int_z p(x|z)p(z)dz$$

The model can be represented using a Bayesian network:



- The Bayesian network represent the joint distribution: $p(x, z) = p(x|z)p(z)$
- We can draw samples from this Bayesian network by first sampling from $p(z)$ and then sampling from $p(x|z)$ to obtain a pair (x, z).
- The above sample of x is from $p(x)$ – the distribution that we want to learn.
- Hence, the key is to learn two distribution: $p(x|z)$ and $p(z)$.

Variational Auto-Encoder (VAE)



	$P(X\|Z)$	
	$Z=0$	$Z=1$
$X=0$	0.1	0.3
$X=1$	0.9	0.7

	$P(Z)$
$Z=0$	0.2
$Z=1$	0.8

The steps:

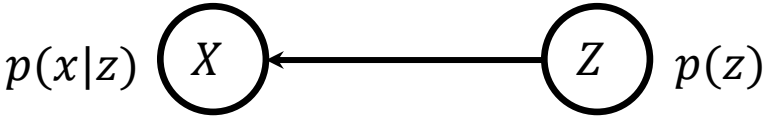
1. Draw a sample z from $p(z)$.
2. Substitute the the above z to $p(x|z)$ and draw a sample x.

For simple case:

Example: suppose that x and z are binary variables and both are observed. We can estimate their distribution tables

When X is a complex variable such as image, and we don't observe Z, we have to approximate $p_\theta(x|z)$ and assume a prior distribution for $p(z)$.

Variational Auto-Encoder (VAE)



The steps:

1. Draw a sample z from $p(z)$.
2. Substitute the the above z to $p(x|z)$ and draw a sample x.

Original VAE:

- Assume: $p(z) = N(0, I)$
- Approximate $p(x|z) \approx p_\theta(x|z)$, where θ is learned from data.

The problem becomes learning $p_\theta(x|z)$, θ is the set of parameters such as parameters in neural networks.

Variational Auto-Encoder (VAE)

How do we learn $p_\theta(x|z)$? ⇒Variational inference

Suppose that we are given n data points: $\boldsymbol{x} = \{x_1, x_2, \dots, x_n\}$,

and their associated latent variables: $\boldsymbol{z} = \{z_1, z_2, \dots, z_n\}$.

The objective function to learn $\boldsymbol{p_\theta(x|z)}$ is the evidence lower bound (ELBO):

$$\log p(\boldsymbol{x}) \geq \underbrace{\sum_{i=1}^{n} \left(E_{z_i \sim q_\phi(z_i|x_i)}[\log p_\theta(x_i|z_i)] - D_{KL}[q_\phi(z_i|x_i)||p(z_i)] \right)}_{\text{The ELBO}}$$

By maximizing the ELBO, we obtain θ and ϕ – and hence obtain $p_\theta(x|z)$.

Variational Auto-Encoder (VAE)

Explaining components of the ELBO:

$$\text{ELBO} = \sum_{i=1}^{n} \left(E_{z_i \sim q_\phi(z_i|x_i)}[\log p_\theta(x_i|z_i)] - D_{KL}[q_\phi(z_i|x_i)||p(z_i)] \right)$$

$q_\phi(z_i|x_i)$ — Encoder

$p_\theta(x_i|z_i)$ — Decoder

$p(z_i)$ — Prior

To learn the model, we can set:

- The encoder = Gaussian distribution whose mean and variance are functions of x_i.
- The prior = standard Gaussian $N(0, I)$
- The decoder: depends on training data; it could be Gaussian, Bernoulli, etc.

Variational Auto-Encoder (VAE)

Explaining components of the ELBO:

$$\text{ELBO} = \sum_{i=1}^{n} \left(E_{z_i \sim q_\phi(z_i|x_i)}[\log p_\theta(x_i|z_i)] - D_{KL}[q_\phi(z_i|x_i)||p(z_i)] \right)$$

$q_\phi(z_i|x_i)$

Encoder

$N(z_i; f_\mu(x_i), e^{f_\sigma(x_i)})$

Neural networks with shared layers

$p_\theta(x_i|z_i)$

Decoder

$N(x_i; g_\theta(z_i), \sigma^2 I)$

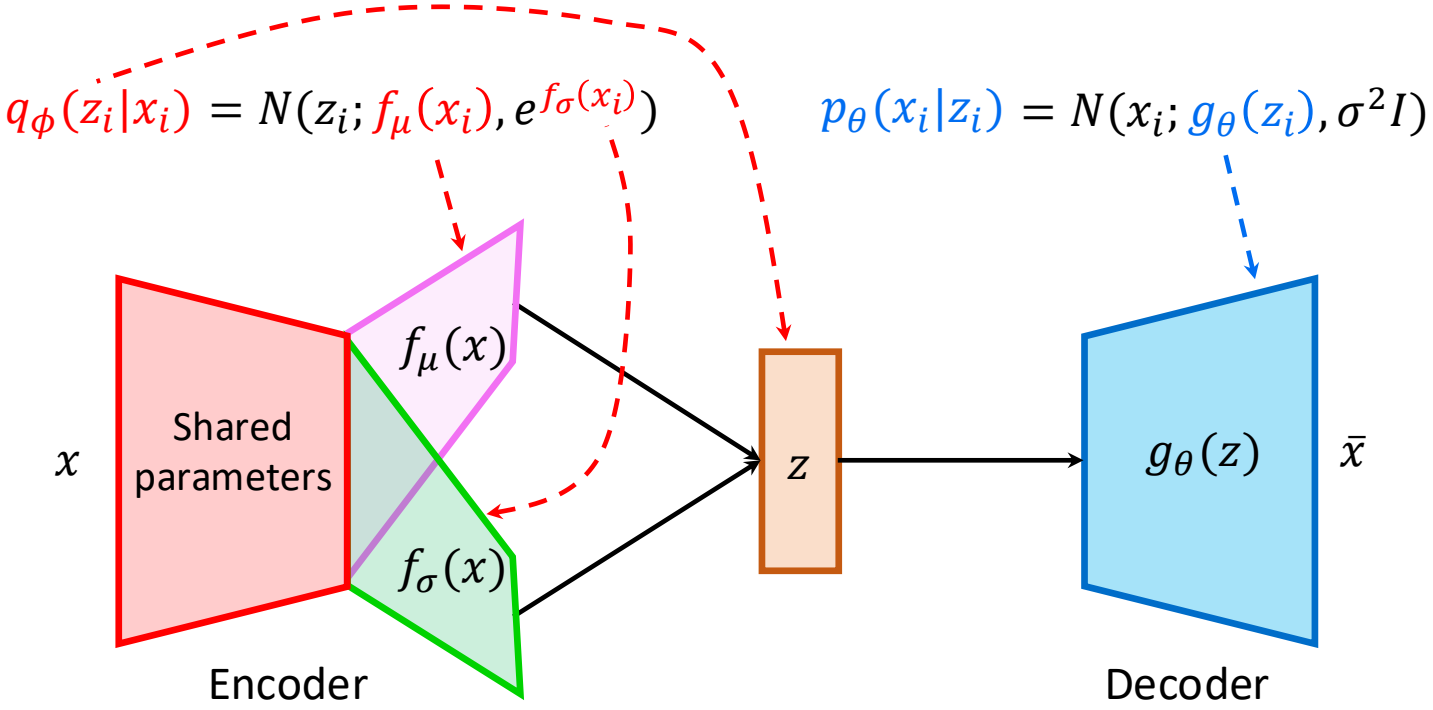
Neural network

$p(z_i)$

Prior

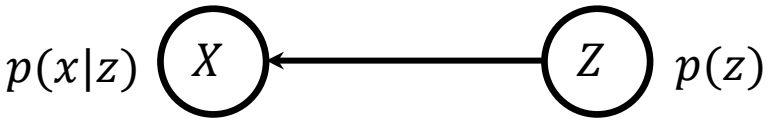
$N(0, I)$

Variational Auto-Encoder (VAE)



Variational Auto-Encoder (VAE)

How to draw samples after training the model?



The steps:

1. Draw a sample z from $p(z) = N(0, I)$.
2. Substitute the the above z to $p_\theta(x|z)$ and draw a sample x.

In many applications, the second step is replaced by calculating the mean of x instead of drawing a sample.

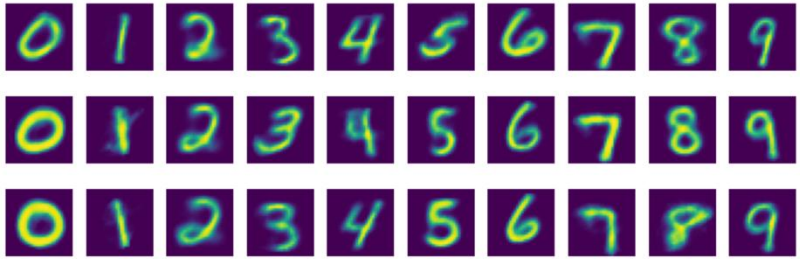
Conditional Variational Auto-Encoder (CVAE)

- The VAE model would learn a distribution $p_\theta(x)$.
- We can condition on a variable y to learn a conditional generative model $p_\theta(x|y)$, where y can be understood as a prompt.
- We replace $q_\phi(z_i|x_i)$ by $q_\phi(z_i|x_i, y_i)$ and $p_\theta(x_i|z_i)$ by $p_\theta(x_i|z_i, y_i)$:

$$\text{ELBO} = \sum_{i=1}^{n} \left(E_{z_i \sim q_\phi(z_i|x_i, y_i)}[\log p_\theta(x_i|z_i, y_i)] - D_{KL}\left[q_\phi(z_i|x_i, y_i) || p(z_i)\right] \right)$$

Conditional Variational Auto-Encoder (CVAE)

Samples drawn from CVAE trained on MNIST dataset:



Samples drawn from CVAE trained on CIFAR10 dataset:

Strength and weaknesses of VAE

Strengths of VAEs

- Efficient Sampling: Sampling is fast once trained, since VAEs generate data in a single forward pass.
- Latent Space Structure: VAEs learn a continuous, structured latent space. This makes them great for interpolation, attribute manipulation, and tasks like representation learning.

Weaknesses of VAEs

- *Blurry Outputs:* VAEs tend to generate blurry or less sharp images, especially compared to GANs or diffusion models. This is due to the assumption of Gaussian likelihood and how the loss is structured.

Diffusion Models

Forward diffusion process
Reverse diffusion process

Diffusion Models

- Currently dominant image/video generation method
- Transform random noise input into image/video, possibly conditioned on language prompt

Video from https://www.youtube.com/shorts/-eQzFLrIAxw

Diffusion Models

The problem:

- We want to draw $x_0 \sim p(x)$.
- We do not know PDF and CDF of x.
- We have data samples of x.

Similar to VAE, each object is associated with a latent variable z.

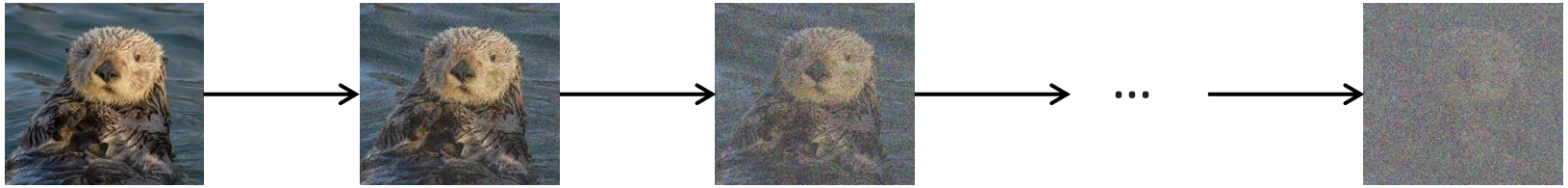
In diffusion models:

The latent z is replaced by multiple latent variables $x_1, x_2, \dots, x_T$ generated from the forward diffusion process

Forward diffusion process

Let $x_0 \sim p(x)$ be a real data distribution.

We add small amount of Gaussian noise to the sample x_0 in T steps, producing a sequence of noisy samples:

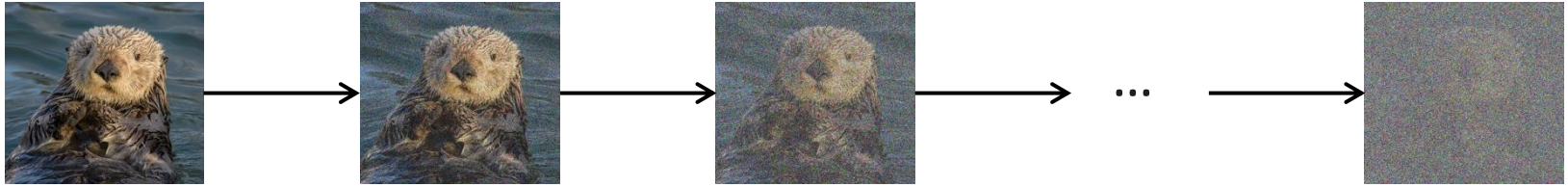


x_0 x_1 x_2 x_T

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \epsilon_{t-1}$$

where $\epsilon_{t-1} \sim N(0, I)$, and $\alpha_t \in (0,1)$

Forward diffusion process



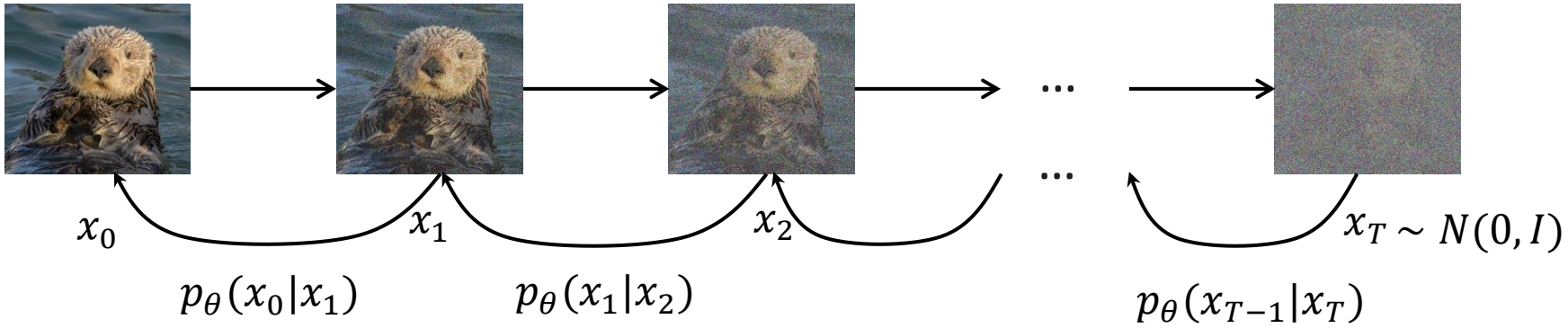
x_0 x_1 x_2 x_T

We can show that (see Appendix in the end of this lecture for details):

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon, \quad \text{where } \bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i \text{ and } \epsilon \sim N(0, I)$$

So, when T is large, $x_T \approx \epsilon \sim N(0, I)$

Reverse diffusion process



If we can reverse the above process, we will be able to recreate the true sample from a Gaussian noise input.

Training diffusion models

How to learn $p_\theta(x_{t-1}|x_t)$?

We maximize the ELBO, similar to VAE:

$$\log p_\theta(x_0) \geq E_q\left[\underbrace{D_{KL}[q(x_T|x_0)||p_\theta(x_T)]}_{L_T} + \sum_{t=2}^{T-1} \underbrace{D_{KL}[q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t)]}_{L_t} - \underbrace{\log p_\theta(x_0|x_1)}_{L_0}\right]$$

For a specific diffusion model, the key is to obtain the form for each L_t.

Training diffusion models

To compute: $L_t = D_{KL}[q(x_{t-1}|x_t, x_0)||p_\theta(x_{t-1}|x_t)]$

From the forward diffusion process, it can be shown that this is a Gaussian distribution.

We parameterize this distribution with neural networks, θ is the set of parameters of the neural network.

- Hence, L_t is KL divergence between two Gaussian distributions.
- L_t is reduced to the loss of predicting the noise in Denoising Diffusion Probabilistic Models (see next slide).

Training diffusion models

Denoising Diffusion Probabilistic Models

Simplified training objective:

$$L_t = E_{t,x_t,\epsilon}\left[\left\|\epsilon - \epsilon_\theta\left(\underbrace{\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon}\right)\right\|_2^2\right]$$

Forward diffusion process: generate noisy x_t(noisy image at a random time step t)

$\epsilon_\theta(\cdot)$ is a function to predict the noise generated from the forward diffusion process.

Strength and weakness of diffusion models

High-Quality Output

- They often generate *very high-fidelity samples*, especially in image generation.

- Competitive or superior to GANs in many benchmarks, especially for large-scale models.

Weaknesses:

- *Slow Sampling*: Sampling is computationally expensive and slow, requiring hundreds to thousands of steps to generate one output. This contrasts with GANs or VAEs, which can often generate samples in a single forward pass.
- *High Computational Cost*: Training large diffusion models requires significant computational resources, especially for large datasets.
- *Memory Intensive*: The large number of steps in the diffusion process can lead to high memory consumption.

Large Language Models

Large Language Models

Large machine learning models trained on broad range of data and can be applied across a wide range of use cases

- Answer questions
- Compose emails
- Make travel plans
- Summarize text
- Chatbot
- Write code
- Prove theorems?
- …

Has taken the world by storm since ChatGPT

Under the Hood

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.c

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

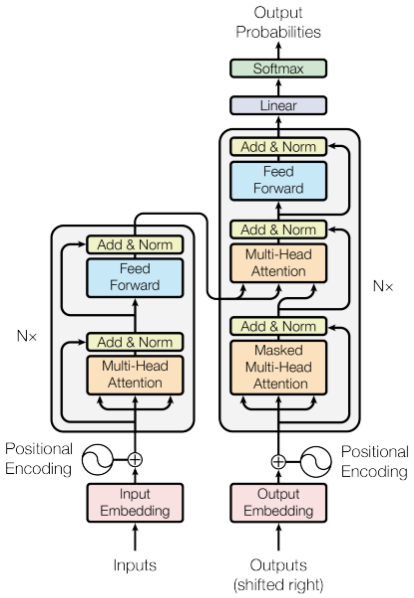
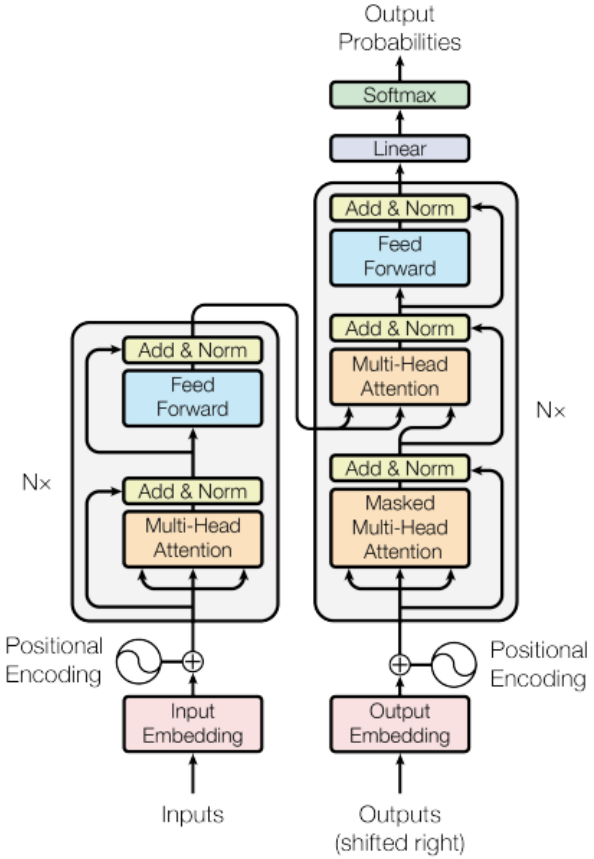
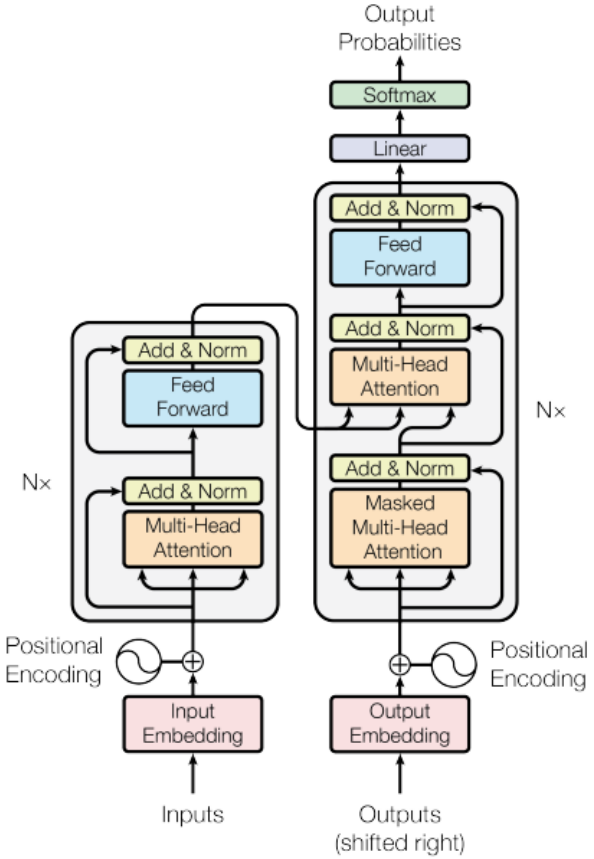


Figure 1: The Transformer - model architecture.

- Usually **transformer**, a deep neural network architecture is used
- The transformer takes as input previously seen tokens to generate the next token
  - Words are tokens in Large Language Models
  - Images can be tokenized by dividing into small subimages



- A **Transformer** is not inherently a generative AI model because it is a general deep learning architecture, not a model with a specific task.
- Transformer is a flexible framework that can be used for multiple tasks, including both discriminative (classification, regression) and generative (text generation, image synthesis) applications



- **Generative AI Models Built on Transformers:**
  - *GPT (Generative Pre-trained Transformer)*: A Transformer-based generative model designed for text generation.
  - *BART & T5*: Use Transformer-based architectures for text-to-text generation.
  - *DALL·E, Stable Diffusion*: Use Transformers for creating images from text descriptions.

Summary

1. Variational Auto-Encoder
   - Inverse Transform Sampling
   - Variational Auto-Encoder
   - Conditional Variational Auto-Encoder
2. Diffusion Models
   - Denoising Diffusion Probabilistic Models
3. Large Language Models
   - Attention Is All You Need

Appendix

Variational Auto-Encoder (VAE)

Variational Auto-Encoder (VAE)

How do we learn $p_\theta(x|z)$? ⇒Variational inference

Suppose that we are given n data points: $\boldsymbol{x} = \{x_1, x_2, \dots, x_n\}$,

and their associated latent variables: $\boldsymbol{z} = \{z_1, z_2, \dots, z_n\}$.

$$\log p(\boldsymbol{x}) \geq \log p_\theta(\boldsymbol{x}) = \log \int p_\theta(\boldsymbol{x}|\boldsymbol{z}) p(\boldsymbol{z}) d\boldsymbol{z}$$

$$\geq \underbrace{\sum_{i=1}^{n} \left(E_{z_i \sim q_\phi(z_i|x_i)}[\log p_\theta(x_i|z_i)] - D_{KL}[q_\phi(z_i|x_i)||p(z_i)] \right)}_{\text{Evidence Lower bound (ELBO)}}$$

Variational Auto-Encoder (VAE)

Maximizing the ELBO w.r.t the parameters θ, ϕ

$$\text{ELBO} = \sum_{i=1}^{n} \left(E_{z_i \sim q_\phi(z_i|x_i)}[\log p_\theta(x_i|z_i)] - D_{KL}\left[q_\phi(z_i|x_i)||p(z_i)\right] \right)$$

$$\approx \sum_{i=1}^{n} \left(\frac{1}{L} \sum_{\ell=1}^{L} \left[\log p_\theta(x_i|z_i^{(\ell)})\right] - D_{KL}\left[q_\phi(z_i|x_i)||p(z_i)\right] \right)$$

$z_i^{(\ell)}$ is a sample drawn from $q_\phi(z_i|x_i)$

This is a function of ϕ and θ

This is a function of ϕ

We cannot compute gradient of the ELBO w.r.t ϕ

Variational Auto-Encoder (VAE)

Reparameterazation:

$$z_i^{(\ell)} \sim q_\phi(z_i|x_i) = N(z_i; f_\mu(x_i), e^{f_\sigma(x_i)})$$

$$\Leftrightarrow z_i^{(\ell)}$$

$$= \underbrace{f_\mu(x_i)}_{\text{The mean}} + \underbrace{\sqrt{e^{f_\sigma(x_i)}}}_{\text{Standard deviation}} \times \epsilon_i^{(\ell)}$$

$$\epsilon_i^{(\ell)} \sim N(0, I)$$

Hence, we have:

$$\text{ELBO} \approx \sum_{i=1}^{n} \left(\frac{1}{L} \sum_{\ell=1}^{L} \left[\log p_\theta(x_i | f_\mu(x_i) + \sqrt{e^{f_\sigma(x_i)}} \times \epsilon_i^{(\ell)}) \right] - D_{KL}\left[q_\phi(z_i|x_i) || p(z_i) \right] \right)$$

Now we can compute gradient of the ELBO w.r.t ϕ

Variational Auto-Encoder (VAE)

The final loss function:

$$\text{ELBO} \approx \sum_{i=1}^{n} \left(\frac{1}{L} \sum_{\ell=1}^{L} \left\| x_i - g_\theta(f_\mu(x_i) + \sqrt{e^{f_\sigma(x_i)}} \times \epsilon_i^{(\ell)}) \right\|_2^2 + \text{KL}_i \right)$$

$$\text{KL}_i = D_{KL}\left[q_\phi(z_i|x_i)||p(z_i)\right]$$

$$= D_{KL}\left[N(z_i; f_\mu(x_i), e^{f_\sigma(x_i)})||N(z_i; 0, I)\right]$$

Diffusion processes

Forward diffusion process

$$x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1-\alpha_t}\epsilon_{t-1}$$
$$= \sqrt{\alpha_t}(\sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1-\alpha_{t-1}}\epsilon_{t-2}) + \sqrt{1-\alpha_t}\epsilon_{t-1}$$
$$= \sqrt{\alpha_t \alpha_{t-1}} x_{t-2} + \sqrt{\alpha_t - \alpha_t\alpha_{t-1}}\epsilon_{t-2} + \sqrt{1-\alpha_t}\epsilon_{t-1}$$

$$\epsilon_{t-2} \sim N(0, I) \Rightarrow \sqrt{\alpha_t - \alpha_t\alpha_{t-1}}\epsilon_{t-2} \sim N(0, (\alpha_t - \alpha_t\alpha_{t-1})I)$$

$$\epsilon_{t-1} \sim N(0, I) \Rightarrow \sqrt{1-\alpha_t}\epsilon_{t-1} \sim N(0, (1-\alpha_t)I)$$

Hence:

$$\sqrt{\alpha_t - \alpha_t\alpha_{t-1}}\epsilon_{t-2} + \sqrt{1-\alpha_t}\epsilon_{t-1} \sim N(0, (\alpha_t - \alpha_t\alpha_{t-1} + 1 - \alpha_t)I)$$
$$\sim N(0, (1 - \alpha_t\alpha_{t-1})I)$$

$$\sqrt{\alpha_t - \alpha_t\alpha_{t-1}}\epsilon_{t-2} + \sqrt{1-\alpha_t}\epsilon_{t-1} = \sqrt{1-\alpha_t\alpha_{t-1}}\bar{\epsilon}_{t-2}$$

Forward diffusion process

$$
\begin{aligned}
x_t &= \sqrt{\alpha_t \alpha_{t-1}} x_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}} \bar{\epsilon}_{t-2} \\
&= \sqrt{\alpha_t \alpha_{t-1} \alpha_{t-2}} x_{t-3} + \sqrt{1 - \alpha_t \alpha_{t-1} \alpha_{t-2}} \bar{\epsilon}_{t-3} \\
&= \cdots \\
&= \sqrt{\alpha_t \alpha_{t-1} \alpha_{t-2} \ldots \alpha_1} x_0 + \sqrt{1 - \alpha_t \alpha_{t-1} \alpha_{t-2} \ldots \alpha_1} \bar{\epsilon}_0
\end{aligned}
$$

Hence, we have:

$$
\boxed{x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \text{where } \bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i}
$$

Training diffusion models

From the forward diffusion process, it can be shown that

$$q(x_{t-1}|x_t, x_0) = \frac{q(x_t|x_{t-1}, x_0)q(x_{t-1}|x_0)}{q(x_t|x_0)} \propto q(x_t|x_{t-1}, x_0)q(x_{t-1}|x_0)$$

$$q(x_{t-1}|x_t, x_0) = N(x_{t-1}; \tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I)$$

$$\tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}(1 - \alpha_t)$$

$$\tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\alpha_t}\,(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} x_t + \frac{\sqrt{\bar{\alpha}_{t-1}}(1 - \alpha_t)}{1 - \bar{\alpha}_t} x_0$$

Hence, L_t is KL divergence between two Gaussian distributions.

Training diffusion models

Denoising Diffusion Probabilistic Models

Se set:

$$\Sigma_\theta(x_t, t) = \sigma_t^2 I \qquad \text{where } \sigma_t^2 = 1 - \alpha_t \text{ or } \sigma_t^2 = \tilde{\beta}_t$$

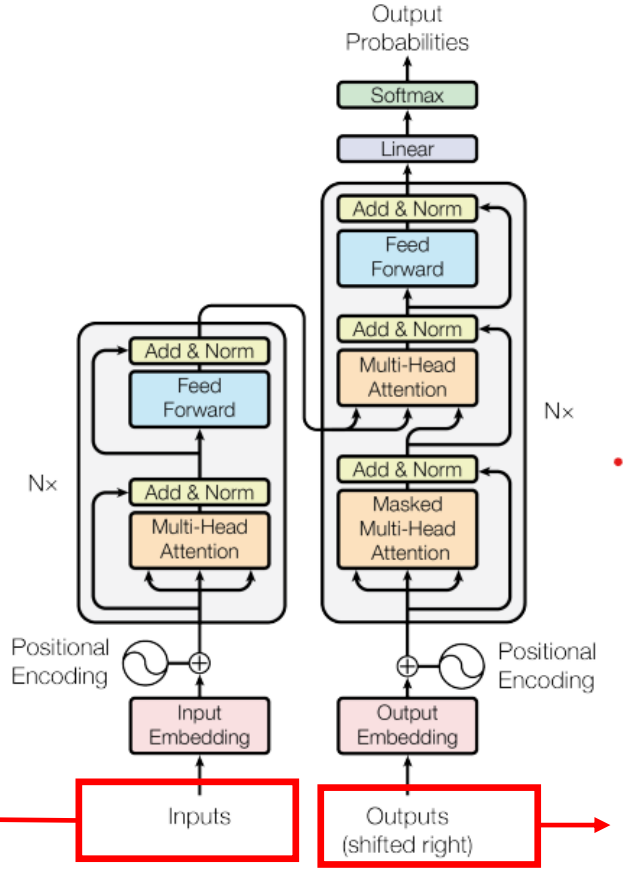
We would obtain:

$$L_t = E_{x_t,\epsilon}\left[\frac{\beta_t^2}{2\sigma_t^2\alpha(1-\bar{\alpha}_t)}\left\|\epsilon - \epsilon_\theta\left(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon\right)\right\|_2^2\right] + \text{constant}$$

Simplified training objective:

$$L_t = E_{t,x_t,\epsilon}\left[\left\|\epsilon - \epsilon_\theta\left(\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon\right)\right\|_2^2\right]$$

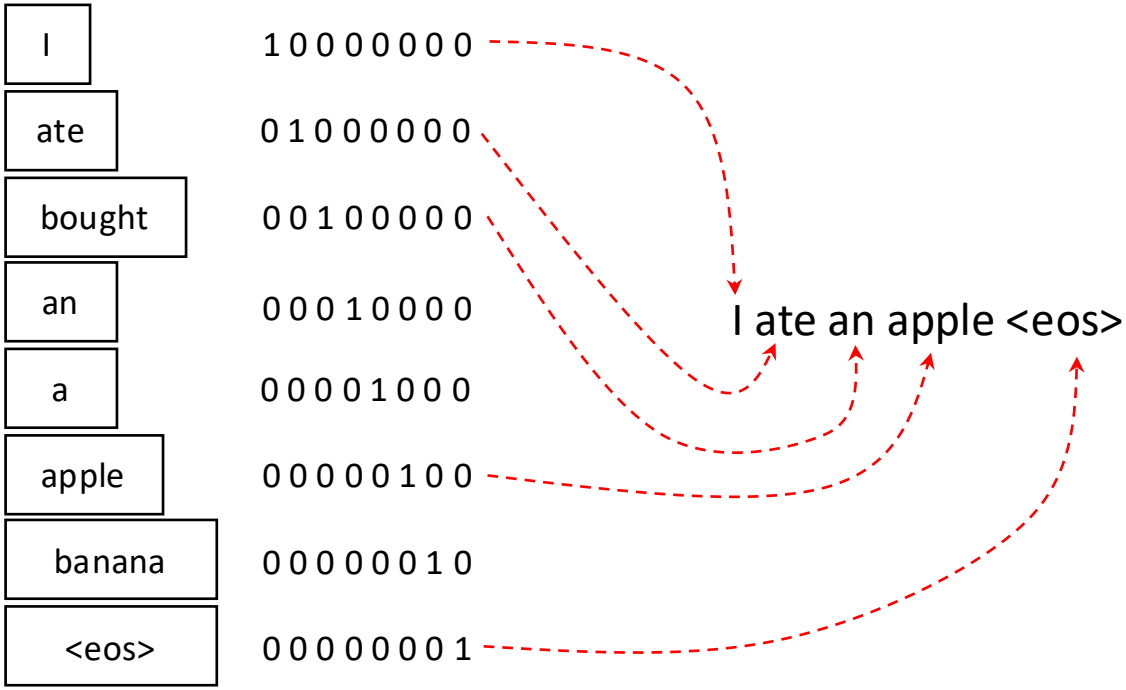
Explaining the components of transformer



I ate an apple

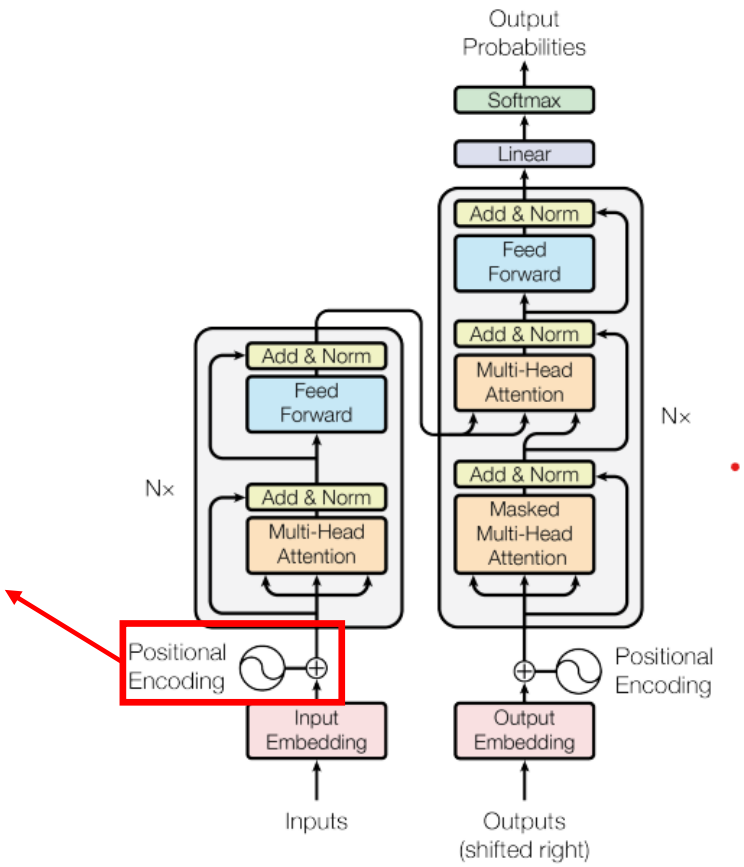
Saya makan sebiji epal

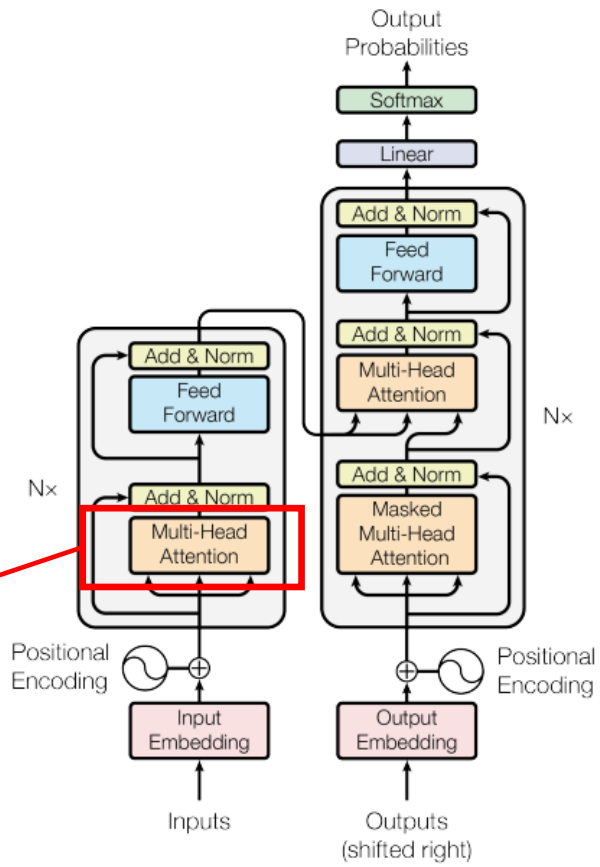
Vocabulary



We use a similar vocabulary for the output.

To ensure that:
“I ate an apple”
is different from:
“An apple ate I”



To calculate the relation between each pair of words:

I ate an apple <eof>